20. Backend Module

20.1 Framework and Rationale

The service is written with FastAPI. Two properties motivated the choice over the alternatives considered.

The first is that validation and documentation derive from the same declaration. A Pydantic model annotated with `Literal["Low", "Middle", "High"]` both rejects a non-conforming request at runtime and appears in the OpenAPI schema as an enumeration. There is no second artefact to keep synchronised, and the interactive documentation at `/docs` cannot describe behaviour the service does not enforce.

The second is that the machine learning module is Python. A Java or Node service would have had to reach the model across a process boundary — through a Python sidecar, or by exporting the Random Forest to ONNX or PMML — and each of those introduces a serialisation format that must agree with the training pipeline about feature ordering and dtype. Given that the preceding sections document five defects arising from exactly this class of disagreement between two representations of the same computation, adding a third representation would have been unwise. FastAPI imports `inference.predict` and calls it as a function.

20.2 Layering

Four layers, enforced by convention rather than by tooling.

Routers (`app/routers/`) parse the request and shape the response. They contain no SQL and no arithmetic. Each router is registered in `main.py` with a tag, which is what groups the endpoints in the generated documentation.

Schemas (`app/schemas/assessment.py`) declare the request and response contracts. All validation lives here and nowhere else; a router never inspects a field value.

Services (`app/services/`) hold the logic. `assessment_service` is the only module that issues database queries, exposing named operations such as `create_assessment`, `save_prediction`, `mark_assessment_failed` and `get_analytics`. `ml_service` is a singleton that wraps the inference module and exposes `predict` and an `is_loaded` property.

Models (`app/models/db_models.py`) declare the five SQLAlchemy entities described in Section 17.

The rule that routers may not touch the session is the one that pays for itself. `POST /assessment` needs to create a row, call the model, persist a prediction and three scores, and mark the assessment complete or failed depending on the outcome. Expressing that as four named service calls keeps the router readable and keeps the transaction boundary in one place.

20.3 Model Loading

`ml_service` imports `inference.predict` once, at module import, which occurs during application startup. The import is wrapped in a broad `try` / `except` that records the failure rather than propagating it, so a service whose model is missing still starts, still answers `/health`, and reports `model_loaded: false`. Requests to `/predict` then raise `RuntimeError`, which the router converts to HTTP 503.

Degrading rather than refusing to boot is deliberate. A container that exits on a missing bind mount produces a restart loop and an opaque failure; a container that starts and reports its own unhealthiness is diagnosable from `/health`.

The module locates the inference package through `ML_PATH`, defaulting to `/ml`, which is where `docker-compose.yml` mounts the `ml/` directory. A fallback path is computed when that directory is absent, and that fallback is wrong: it ascends four directory levels from `app/services/` and lands on the repository's parent rather than its root. The resulting path does not exist. The defect is masked under Docker Compose, where `/ml` is always present, and surfaces only when the service is run directly from a shell without `ML_PATH` set, in which case every prediction returns 503. Section 16.7 records it.

20.4 The Prediction Path

`POST /assessment` proceeds as follows. The body is validated against `AssessmentInput`. `assessment_service.create_assessment` writes a row with `status = "pending"` and commits, so that the attempt is durable before the model is invoked. `ml_service.predict` is called with the validated dictionary. On success, `save_prediction` writes one `predictions` row and three `behavioral_scores` rows and commits them together, and the assessment is marked `completed`. On any exception the assessment is marked `failed` and HTTP 500 is returned.

Committing the assessment row before the prediction is what makes the `failed` status meaningful. Had both been written in one transaction, a failed prediction would roll back the assessment and leave no record that the attempt occurred.

`POST /predict` performs the same validation and the same model call, and writes nothing.

20.5 Persistence

SQLAlchemy 2.0 declarative mapping backs five tables. Sessions are supplied to routers through a `get_db` dependency that yields a session and closes it in a `finally` block, so a session is released whether or not the handler raised.

The engine is constructed from `DATABASE_URL`. When that string names SQLite the engine receives `check_same_thread: False`, and a connection event handler issues `PRAGMA journal_mode=WAL` to permit concurrent readers alongside a writer. Neither adjustment applies to PostgreSQL.

Dialect divergence is handled in exactly one place. The daily-count aggregation in `get_analytics` inspects `db.bind.dialect.name` and emits `strftime` or `to_char` accordingly, because no portable spelling of date truncation exists across the two backends.

20.6 Middleware and Logging

A single middleware wraps every request. It records method, path, status and elapsed milliseconds to the application log, then attempts to write an `audit_logs` row.

Two characteristics of this middleware deserve to be stated plainly rather than glossed.

It is described in its own comment as a "non-blocking best-effort" write. It is best-effort — the write is wrapped in a bare `except` so that a logging failure cannot fail the request. It is not non-blocking. `dispatch` is a coroutine, and it opens a synchronous SQLAlchemy session and commits without awaiting, which occupies the event loop for the duration of the write. Under concurrency this serialises requests behind the audit table.

It persists the full request body. For `POST /assessment` that body is a complete financial disclosure: income, expenses, debt, credit score. These are written to a table that no endpoint exposes and no policy protects, in a system with no authentication. On synthetic inputs this is harmless. Section 26 treats it as a limitation rather than a feature.

20.7 Security Posture

There is none, and the report should not pretend otherwise.

`SECRET_KEY` is declared in the settings model and consumed by no code path. No authentication middleware exists, no route carries a dependency that inspects a credential, and the `users` table is never written. `GET /assessments` returns every stored assessment to any caller without pagination limits beyond a `limit` parameter capped at 100. CORS is restricted to two localhost origins, which is the only access control present, and it constrains browsers rather than clients.

This was a scoping decision rather than an oversight — authentication was outside the objectives listed in Section 7 — but its consequences are load-bearing for how the system may be used. The application is a demonstration operating on synthetic data. It is not deployable against real financial disclosures without an authentication layer, an authorisation model for the history and analytics endpoints, and a policy governing what the audit log retains.

> **Figure 20.1** — *Backend package structure,* as a layered block diagram: routers → schemas → services → models, with `ml_service` shown calling out to the mounted `ml/` directory and `assessment_service` calling the database. Place in Section 20.2.
>
> **Screenshot 20.1** — *Backend startup log,* showing `Database initialized.` followed by `ML models loaded and ready.` This is the evidence that the bind mount resolved and the artefacts deserialised. Place in Section 20.3.
>
> **Figure 20.2** — *State transitions of `Assessment.status`:* pending → completed, and pending → failed. A two-transition diagram. Place in Section 20.4 to support the argument about the early commit.